Financial RAG comparison report

Customer Case Intelligence | Fictional demonstration

Financial document intelligence: chunking and reranking

Customer Case Intelligence | Return & Refund Intelligence

One case. Every connection.

Submission status

Measured local retrieval comparisons are complete. Live Pinecone synchronization, all 24 case/role checks and a controlled local-fallback test passed. RAGAS execution status: complete. See the RAGAS companion report for detailed completion counts.

This report separates measured local/cloud retrieval results from RAGAS answer-quality judgments. A final demo recording and hosting deployment remain separate deliverables.

Required framework

Field	Decision
Use case	A support specialist asks where a fictional device return or refund stalled and which evidence connects merchant and bank events, through a case-scoped web assistant.
Corpus	91 English PDFs: synthetic orders, payments, returns, carrier scans, inspections, refund ledgers, statements, disputes and conversations, plus attributed public-guidance paraphrases. Source PDFs and their hash manifest are the narrative source of truth; structured fictional records support deterministic reconciliation.
Ingestion and cleaning	Extract PDF text and tables with page/source provenance, normalize extraction artifacts and preserve financial table headers. Private uploads are supplemental and cannot overwrite verified case balances.
Ingestion and freshness	The demonstration corpus is a versioned snapshot with explicit PDF rebuild and index synchronization; there is no live institution feed or production freshness SLA. New private PDFs are parsed on upload and enter only that owner's local case workspace.
Chunking and embedding	Fixed recursive chunks target 350 cl100k_base tokens with 40 overlap; semantic splits use the 70th-percentile adjacent sentence/paragraph embedding distance and the same cap. BGE-small-en-v1.5 provides 384-dimensional local embeddings for both documents and queries.
Retrieve	BM25 and cosine dense retrieval are fused using reciprocal ranks into the same top-20 candidate pool, returning six chunks in the controlled comparison. Dense retrieval uses optional Pinecone with local fallback; private uploads remain local.

Controlled retrieval comparison

Explicit cl100k_base tokenizer. 350-token recursive baseline (40 overlap); semantic 70th-percentile adjacent embedding distance, same 350-token cap; BGE embeddings + BM25 RRF top-20 shortlist in every condition; return six chunks; no parent-document promotion during evaluation.

The same 32 development questions and frozen sufficient-evidence anchors are used in every configuration. Exact source ID and whitespace-normalized passage anchors determine relevance. These are retrieval metrics, not RAGAS judge scores.

Configuration	Chunks	Recall@6	Precision@6	MRR@6	Median ms
fixed	274	0.875	0.172	0.683	15.6
Fixed + CE	274	0.891	0.172	0.743	2283.1
semantic	989	0.828	0.151	0.593	12.1
Semantic + CE	989	0.859	0.161	0.610	1697.8
Table-aware + CE	342	0.844	0.161	0.716	2550.4

Pinecone retrieval comparison

The same frozen 32 questions were evaluated with strict Pinecone dense search plus local BM25. No local dense fallback is allowed in this experiment; these are retrieval measurements, separate from RAGAS generation/judge scores.

Configuration	Chunks	Recall@6	Precision@6	MRR@6	Median ms
fixed	274	0.875	0.172	0.683	204.9
Fixed + CE	274	0.891	0.172	0.743	2041.2
semantic	989	0.828	0.151	0.593	202.4
Semantic + CE	989	0.859	0.161	0.610	1285.2
Table-aware + CE	342	0.844	0.161	0.716	2947.3

Pinecone fixed reranking: evidence recall +1.56 percentage points; MRR +0.060.

Pinecone semantic reranking: evidence recall +3.12 percentage points; MRR +0.017.

Cloud and earlier local timings were measured at different times under different loads; they do not establish an isolated backend speed comparison. Per-query cloud backend labels and corpus hashes are in evaluation-pinecone.json.

Reranking impact

The trained cross-encoder is Xenova/ms-marco-MiniLM-L-6-v2, run locally with FastEmbed ONNX. It scores query-passage pairs jointly after hybrid retrieval. All paired conditions use the same candidate pool; source title and case metadata accompany passages in every reranked condition.

- fixed: recall +1.56 percentage points; precision +0.00 points; MRR +0.060; median latency +2267.5 ms.
- semantic: recall +3.12 percentage points; precision +1.04 points; MRR +0.017; median latency +1685.8 ms.

Reranking improved average recall modestly and added substantial local CPU latency. Semantic chunking did not beat the fixed baseline on this dataset. A separate eight-question financial-table probe passed all eight questions in each tested configuration; this small synthetic test is not a general table-accuracy guarantee.

Pinecone storage and fallback

A serverless external-vector index must use 384 dimensions and cosine similarity. Separate content-versioned namespaces isolate fixed, semantic and table-aware chunks; deterministic vector IDs make re-uploading idempotent. Cloud records contain the vector, chunk text, source/page, case scope and required role level. Full PDFs, chunk metadata and NumPy vectors remain local for provenance and fallback.

Pinecone case/role filters are applied before dense ranking. Every returned ID is checked against the local permitted set, and no remote text is trusted as authoritative. Cloud failure, incomplete synchronization or failed validation uses local retrieval. Strict Pinecone evaluation refuses fallback, preventing mislabeled cloud results. Private owner uploads are never included in the cloud synchronization command.

Local storage: data/corpus/*.pdf and manifest.json hold source material; data/index/*.chunks.json holds chunk text/metadata; data/index/*.npy holds vectors. work/uploads holds private originals/extracted records. Runtime identities, keys, uploads, indexes and caches are excluded from Git and submission archives.

RAGAS evaluation

Four configurations use the same 32 manually authored question/reference pairs: fixed, fixed with cross-encoder, semantic, and semantic with cross-encoder. The controlled LangChain generation chain receives only retrieved passages, with the same structured answer prompt and quote checks across conditions; additional deterministic application reconciliation is withheld to isolate retrieval effects.

RAGAS 0.3.9 evaluates Faithfulness, LLMContextPrecisionWithReference and LLMContextRecall. Generation and judging use separately invoked OpenAI models recorded in the JSON artifact. RAGAS metrics are model judgments and are not interchangeable with exact-span recall/precision. Telemetry and tracing are disabled for this command.

Faithfulness grades the complete rendered response, including factual claims, unknowns and the suggested next step. Reference-guided context precision measures rank-weighted usefulness, while context recall attributes reference-answer claims to retrieved passages. Recommendations can therefore affect faithfulness even when every retained claim has an exact source quote; quote matching is not a semantic-grounding guarantee.

Generation model: gpt-4.1-mini; judge model: gpt-4.1-mini; temperature 0 for both. Current status: complete. The table records measured judge outputs when available; missing, failed or non-finite metrics remain unmeasured, never fabricated zeros.

Configuration	Completed rows / expected	Faithfulness	Context precision	Context recall
fixed	32 / 32	0.784 (n=32)	0.849 (n=32)	0.984 (n=32)
Fixed + CE	32 / 32	0.735 (n=32)	0.884 (n=32)	0.984 (n=32)
semantic	32 / 32	0.676 (n=32)	0.863 (n=32)	0.953 (n=32)
Semantic + CE	32 / 32	0.592 (n=32)	0.838 (n=32)	0.969 (n=32)

RAGAS paired impact

- fixed, faithfulness: -0.049 (n=32).
- fixed, context_precision: +0.034 (n=32).
- fixed, context_recall: +0.000 (n=32).
- semantic, faithfulness: -0.084 (n=32).
- semantic, context_precision: -0.026 (n=32).
- semantic, context_recall: +0.016 (n=32).

Interpretation of the measured RAGAS results

Across the six paired RAGAS comparisons, reranking improved 2 metric averages, reduced 3, and left 1 unchanged. Retrieval improvements do not guarantee more faithful generated answers; the score table must be read alongside latency and individual failures.

The application retains fixed-size hybrid retrieval by default and optional deeper reranking. These development-set measurements support reviewing the trade-off rather than declaring semantic chunking or reranking universally superior. A held-out real-document study and repeated judge runs are needed before a production choice.

Lowest-faithfulness examples for review

- semantic, CASE-1001: What is the requested return amount? Faithfulness 0.000. Review the full response, unknowns, recommendation and retrieved source IDs in the JSON artifact; this is a judge flag, not proof that every factual claim is false.
- Semantic + CE, CASE-1001: What is the requested return amount? Faithfulness 0.000. Review the full response, unknowns, recommendation and retrieved source IDs in the JSON artifact; this is a judge flag, not proof that every factual claim is false.
- Semantic + CE, CASE-1002: What amount was approved? Faithfulness 0.000. Review the full response, unknowns, recommendation and retrieved source IDs in the JSON artifact; this is a judge flag, not proof that every factual claim is false.
- semantic, CASE-1008: What payment was captured? Faithfulness 0.200. Review the full response, unknowns, recommendation and retrieved source IDs in the JSON artifact; this is a judge flag, not proof that every factual claim is false.

All 128 rows have valid values for all three metrics. Requested retrieval backend: pinecone; actual per-hit backend labels are retained in the JSON artifact. Checkpoint retries retained successful rows; failed attempts are excluded from score averages and can add provider usage.

Reproduce and complete

1. Run scripts/configure_providers.py locally to enter Pinecone and OpenAI keys without echo. It writes only the ignored permission-600 .env file. Hosted infrastructure keys belong in server-side secret settings, never frontend code or a report.

2. Create the compatible Pinecone index, then run scripts/sync_pinecone.py. Confirm every strategy namespace is fully visible. Restart the application after environment changes.

3. Install requirements-eval.txt (or the exact requirements-eval.lock snapshot). Run scripts/evaluate_ragas.py --preflight, then scripts/evaluate_ragas.py --backend pinecone. For a controlled local experiment use --backend local. The full comparison evaluates 128 answers plus multiple judge calls and incurs provider charges; failed attempts and retries can add calls.

4. Run scripts/build_comparison_report.py --pdf after live evaluations to refresh this document. Run scripts/scan_publication.py and scripts/package_submission.py before sharing; record the application demo separately.

Limits and honest interpretation

- Eight fictional cases and template-assisted source documents cannot establish production accuracy on real bank or merchant records.
- The 32 questions are a development benchmark, not an independent held-out test. Reference annotations may miss valid supporting paraphrases.
- Existing retrieval timing measurements exclude downloads, embedding/index construction and OpenAI generation; concurrent local workloads limit timing precision.
- A cross-encoder can only reorder retrieved candidates. It cannot recover a passage outside the candidate set or prove a real-world event occurred.
- Same-family generation/judge models can share biases. This is a single-pass evaluation with stochastic model outputs, not a confidence-interval estimate. RAGAS results require human review, especially for claims about missing credits or provisional adjustments.
- No live bank transfer, refund, dispute submission, external ticket or customer message is executed. The optional bank sandbox adapter is not a verified live institution connection.

Implementation provenance

The supplied academy solution kit is architectural reference only. No Apple filings, sample scores, proprietary code or sample prompts are copied. This original implementation keeps the required fixed/semantic comparison, cross-encoder and RAGAS layers while documenting its different corpus, model sizes, chunk settings and hybrid retrieval.

Primary technical references: https://sdk.pinecone.io/python/how-to/vectors/upsert-and-query.html ; https://docs.ragas.io/en/v0.3.9/getstarted/evals/ ; https://developers.openai.com/api/docs/guides/production-best-practices .

Detailed provenance and per-query retrieval failures are in docs/evaluation-pdf.json and .md. Per-query RAGAS scores, completion state, model names and corpus/reference/prompt hashes are in docs/evaluation-ragas.json. No key or private customer record belongs in these artifacts.